

27. Accessory Authentication

Accessory authentication provides a mechanism for devices to trust the identities and supported features of specification compliant accessories.

Accessory authentication requires an Apple Authentication Coprocessor, a hardware component containing an X.509 certificate and the ability to generate responses to authentication challenges.

An authentication *challenge* is a random value sent from a device to an accessory. The accessory shall perform a computation on the offered challenge and return a result, that is, the *challenge response*, to the device for verification.

This feature is supported on tvOS 10.2 or later.

27.1 Requirements

Accessory hardware shall incorporate an [Apple Authentication 3.0 Coprocessor](#) (page 696). Earlier versions of the Apple Authentication Coprocessor are not allowed.

Accessory hardware designs may support multiple package versions of the [Apple Authentication 3.0 Coprocessor](#) (page 696) for additional component sourcing/manufacturing flexibility.

An accessory may use one authentication processor to authenticate itself to more than one device. Conversely, multiple accessories shall not share one Apple Authentication Coprocessor or authenticate on behalf of each other to the same device.

The accessory shall have successfully established an iAP2 link over an iAP2 transport.

Requirements differ when using control session version 1 or control session version 2, see [Control Session](#) (page 476). When using control session version 2, the authentication messages shall be included in the [IdentificationInformation](#) (page 872) message.

Accessories supporting the Accessory Authentication (control session version 1) feature using iAP2 shall send or receive the following iAP2 control session message(s):

- [RequestAuthenticationCertificate](#) (page 870)
- [AuthenticationCertificate](#) (page 870)
- [RequestAuthenticationChallengeResponse](#) (page 870)
- [AuthenticationResponse](#) (page 871)

[AuthenticationFailed](#) (page 871)

[AuthenticationSucceeded](#) (page 871)

Accessories supporting the Accessory Authentication (control session version 2) feature using iAP2 shall send or receive the following iAP2 control session message(s):

[RequestAuthenticationCertificate](#) (page 870)

[AuthenticationCertificate](#) (page 870)

[RequestAuthenticationChallengeResponse](#) (page 870)

[AuthenticationResponse](#) (page 871)

[AuthenticationFailed](#) (page 871)

[AuthenticationSucceeded](#) (page 871)

[AccessoryAuthenticationSerialNumber](#) (page 872)

27.2 Usage

Authentication is always initiated by transmission of a [RequestAuthenticationCertificate](#) (page 870) message from a device to an accessory.

If control session version 2 is in use and the [RequestAuthenticationCertificate](#) (page 870) message contains a [RequestAuthenticationCertificateSerialNumber](#) parameter, the accessory should retrieve the X.509 certificate serial number from its Apple Authentication Coprocessor and transmit it to the device using an [AccessoryAuthenticationSerialNumber](#) (page 872) message.

If the serial number is not available or the [RequestAuthenticationCertificateSerialNumber](#) parameter is not present, the accessory shall retrieve the X.509 certificate from its Apple Authentication Coprocessor and transmit it to the device using an [AuthenticationCertificate](#) (page 870) message. The certificate shall be sent within 1 second of receiving an [RequestAuthenticationCertificate](#) (page 870) message.

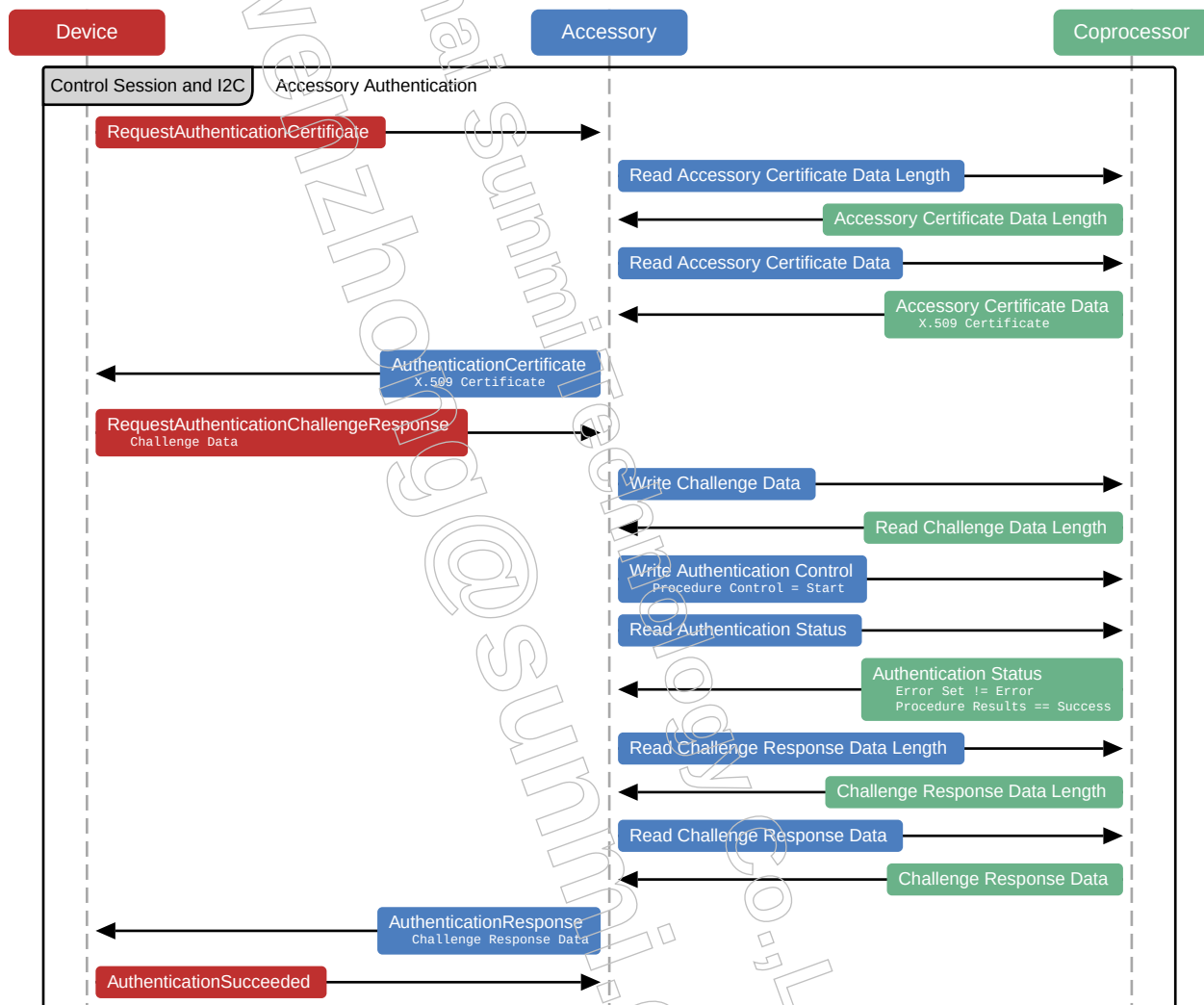
If the certificate is valid, the device responds with a [RequestAuthenticationChallengeResponse](#) (page 870) message. The accessory shall use the Apple Authentication Coprocessor to compute a response to the authentication challenge and send an [AuthenticationResponse](#) (page 871) message with a response. This response shall be sent within 2 seconds of receiving the [RequestAuthenticationChallengeResponse](#) (page 870) message.

If the authentication response is correct, the device will send an [AuthenticationSucceeded](#) (page 871) message. The accessory shall immediately proceed to [Accessory Identification](#) (page 281) unless it is using control session version 2 and has already identified. With control session version 2, once the device starts authentication, the accessory shall complete the process within 15 seconds.

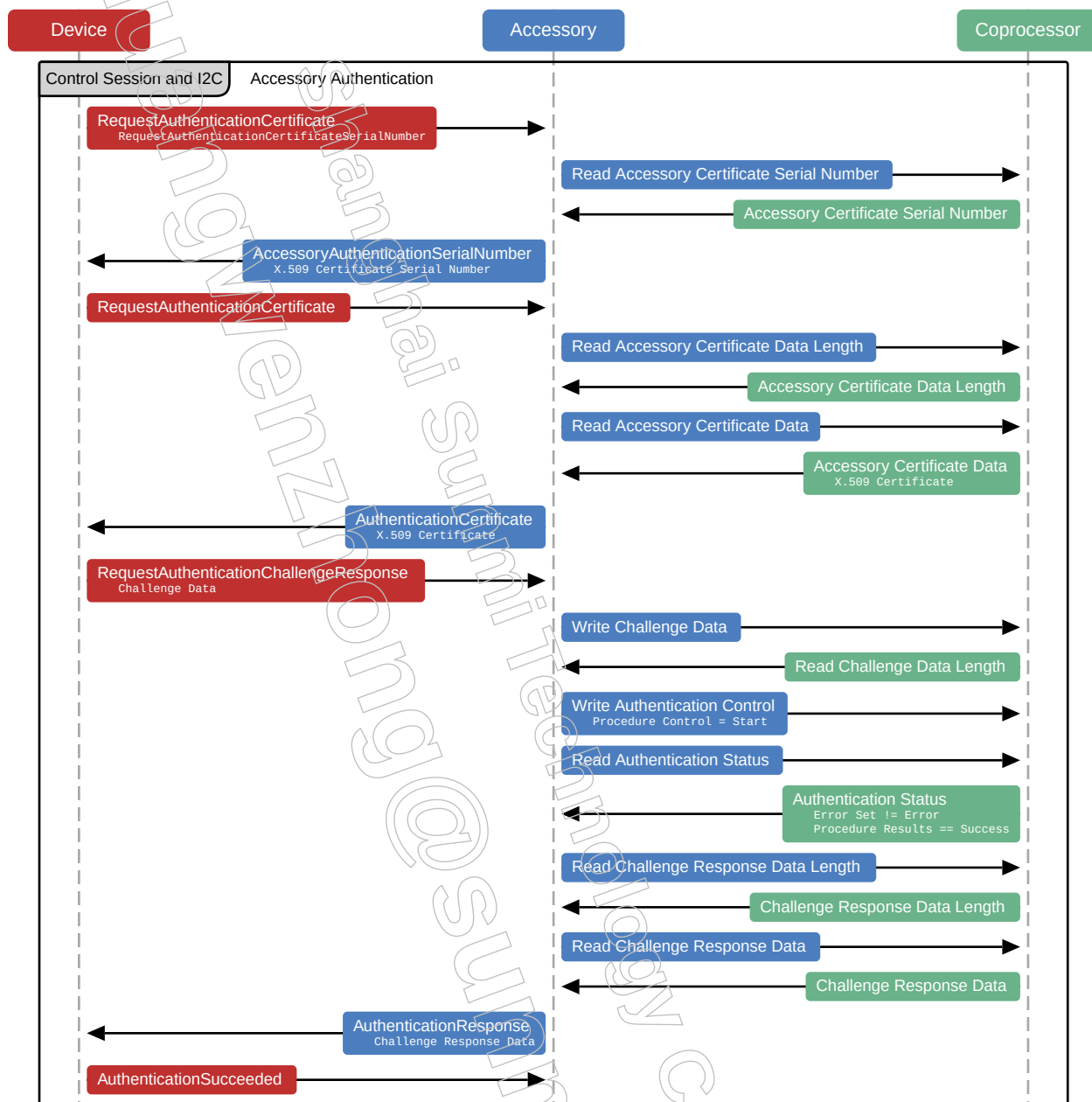
To demonstrate correct handling of an [AuthenticationFailed](#) (page 871) message during self-certification, accessories shall send an empty or invalid certificate when the Apple Authentication Coprocessor is not present.

27.3 Examples

27.3.1 Typical Accessory Authentication (control session version 1)



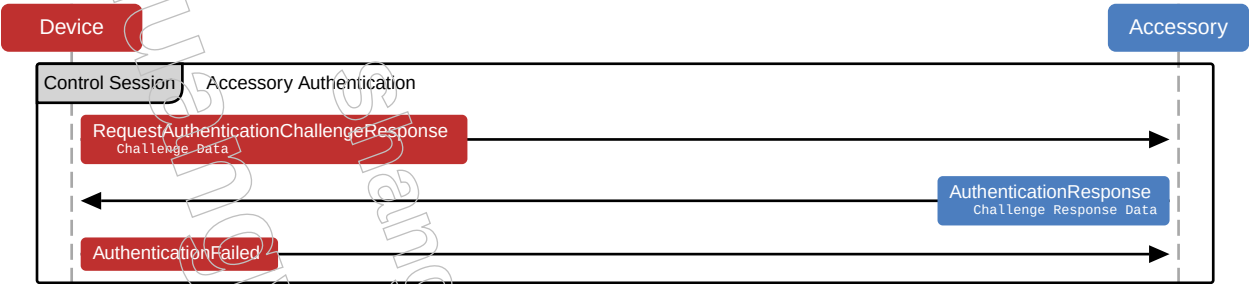
27.3.2 Typical Accessory Authentication (control session version 2)



27.3.3 Accessory Authentication Failure Due To Invalid Certificate



27.3.4 Accessory Authentication Failure Due To Invalid Response



28. Accessory Identification

All accessory interface features other than charging require the identification feature to be supported.

28.1 Requirements

Accessories shall have successfully completed [Accessory Authentication](#) (page 276) unless using control session version 2. See [Control Session](#) (page 476).

Control session version 2 allows the device to choose the order of identification and authentication, so the accessory shall be ready for either [StartIdentification](#) (page 872) or [RequestAuthenticationCertificate](#) (page 870) after successful link layer negotiation.

Accessories supporting the Accessory Identification feature using iAP2 shall send or receive the following iAP2 control session message(s):

- [StartIdentification](#) (page 872)
- [IdentificationInformation](#) (page 872)
- [IdentificationAccepted](#) (page 878)
- [IdentificationRejected](#) (page 878)
- [CancelIdentification](#) (page 879)

Accessories supporting the Accessory Identification feature using iAP2 may also send or receive the following iAP2 control session message(s):

- [IdentificationInformationUpdate](#) (page 879)

28.2 Usage

Identification is always initiated with the transmission of a [StartIdentification](#) (page 872) message from the device to the accessory. The accessory shall respond with an [IdentificationInformation](#) (page 872) message. The [IdentificationInformation](#) (page 872) message is evaluated by the device to determine whether all of the requested iAP2 connection features are supported. If the parameters are valid and the device can support all of the requested messages, the device sends an [IdentificationAccepted](#) (page 878) message to the accessory.

Otherwise, an [IdentificationRejected](#) (page 878) message is sent to the accessory detailing any invalid parameters and unsupported messages present in the previous [IdentificationInformation](#) (page 872) message.

Note:

It is possible for the list of unsupported messages to be empty; which is an indication of incorrect accessory implementation, rather than unsupported features on a specific device.

ATS Traffic Sniffer output and device console logs, see <https://developer.apple.com/documentation/xcode/acquiring-crash-reports-and-diagnostic-logs>, provide additional supporting information useful for accessory development. Apple strongly recommends accessory developers pay close attention to both ATS output and device logs.

If the identification information has been rejected, the accessory shall send another [IdentificationInformation](#) (page 872) message with different parameters or abort the identification process using [CancelIdentification](#) (page 879). Cancellation of the identification process causes the device to inform the user the accessory is not supported.

Failure of the accessory to send [IdentificationInformation](#) (page 872) within 1 second of either [StartIdentification](#) (page 872) or [IdentificationRejected](#) (page 878) is grounds for failing to pass self-certification.

Once [IdentificationAccepted](#) (page 878) is received, the accessory shall not send any more identification messages other than [IdentificationInformationUpdate](#) (page 879).

It is possible for an accessory to update its identification information after an [IdentificationAccepted](#) (page 878) message by sending an [IdentificationInformationUpdate](#) (page 879) message. For example, the user may change the accessory's active language. When that occurs, the accessory shall send an [IdentificationInformationUpdate](#) (page 879) message containing both the new active language and accessory information strings in the new language.

If the identification process fails or the accessory cancels identification, the device may choose to restart the identification process or terminate the iAP2 link.

28.2.1 Manufacturing Information

Accessories shall provide values for the following [IdentificationInformation](#) (page 872) message parameters matching the accessory packaging information:

- Name
- ModelIdentifier
- Manufacturer
- SerialNumber

Additionally, the `FirmwareVersion` and `HardwareVersion` parameters shall uniquely reflect the current versions of the accessory's firmware and hardware respectively.

Accessories shall not provide empty strings or generic string values (such as those provided by a reference design or component vendor) for any manufacturing information parameters.

The `SerialNumber` parameter should be unique for each accessory (for example, serialized); because devices may use this to differentiate multiple accessories of the same type.

If an accessory allows users to customize Name parameters, the accessory shall provide a means to restore the factory default name.

28.2.2 Sent/Received iAP2 Control Session Messages

The `MessagesSentByAccessory` and `MessagesReceivedFromDevice` parameters in [IdentificationInformation](#) (page 872) are critical to accessory identification. Accessories shall exhaustively enumerate all iAP2 control session messages they can send and receive. During the self-certification process, the accessory shall be used in a manner demonstrating the correct implementation of all declared messages.

There is only one exception to this requirement: Accessories using control session version 1 do not need to register Authentication and Identification features in the `MessagesSentByAccessory` or `MessagesReceivedFromDevice` lists.

28.2.3 Power Capabilities

Accessories shall provide values for `MaximumCurrentDrawnFromDevice` and `PowerProvidingCapability` parameters in their [IdentificationInformation](#) (page 872) messages.

If an accessory does not draw power from the device, `MaximumCurrentDrawnFromDevice` shall be set to 0 mA.

If an accessory does not provide power to the device or does not use iAP2 to declare its power providing capability, `PowerProvidingCapability` shall be set to `None`.

Otherwise, the accessory shall set the parameters as required by [Accessory Power \(Lightning\)](#) (page 295) and [Device Power \(Lightning\)](#) (page 328).

28.2.4 iAP2 Transport Components

Accessories implementing iAP2 shall identify at least one transport component supporting an iAP2 connection. The iAP2 supporting transport component shall contain a `TransportSupportsIAP2Connection` parameter. Depending on the type of transport, additional identifying information, such as a Media Access Control (MAC) address, may be required. The iAP2 connection shall be implemented using the identified transport component.

Some accessory interface features require a specific transport component to be implemented in order to function. This requirement is independent of whether or not an iAP2 connection is implemented on the transport. For example, an accessory shall identify a `USBDeviceModeTransportComponent` for digital audio over the [USB Device Mode](#) (page 513) transport. Every identified transport component shall have a unique identifier and human-readable name reflecting the intended purpose of the component, such as 'iAP2' or 'Digital Audio Speakers'.

All transport components shall have unique identifiers.

28.2.5 Product Plan UID

Accessories shall provide a value for the `ProductPlanUID` parameter in the [IdentificationInformation](#) (page 872) message matching the value assigned by the MFi Portal.

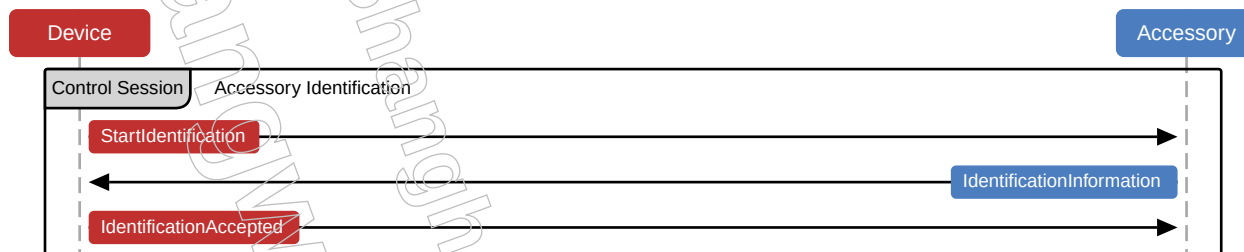
The Product Plan UID is a 16 character unique identifier for the product plan associated with the accessory. The value is available in the product plan header in the MFi Portal and is different from the Product Plan ID.

28.2.6 Additional Parameters

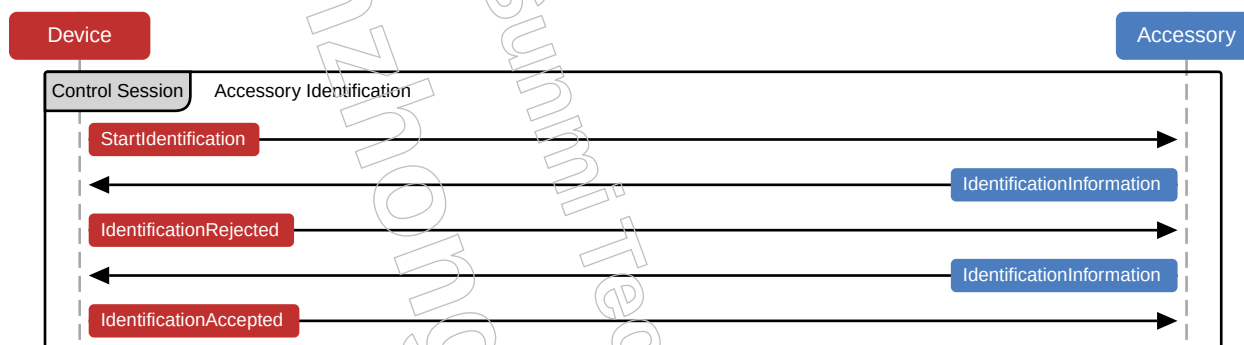
Several accessory interface features require the accessory to supply additional identification parameters. See the appropriate sections in the specification for details.

28.3 Examples

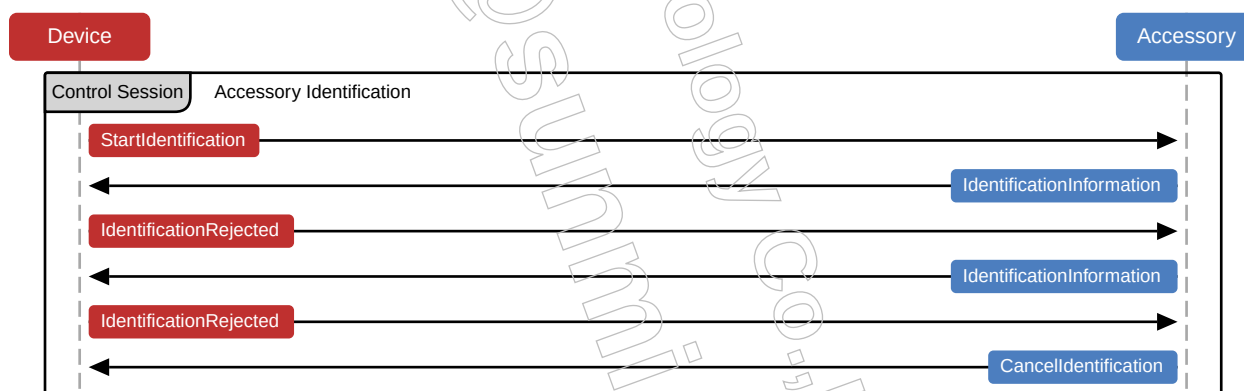
28.3.1 Typical Accessory Identification



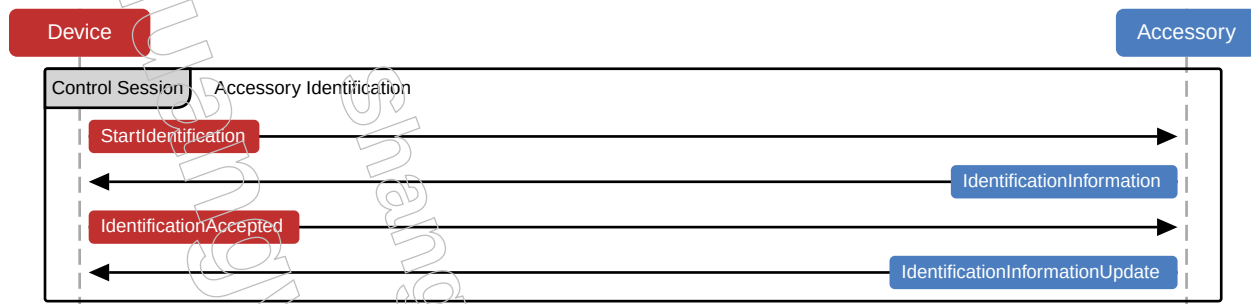
28.3.2 Successful Accessory Identification With Two Tries



28.3.3 Unsuccessful Accessory Identification After Two Tries



28.3.4 Accessory Identification With Information Update



28.4 Test Procedures

Equipment

- Latest revision of ATS software
 - ATS hardware
1. Verify the accessory correctly provides values for the accessory information fields in the ATS iAP Summary.
 - a. Connect the accessory and the device to ATS.
 - b. View the iAP Summary page.
 - c. Verify fields (name, brand, etc.) are correctly populated with no "filler" entries, such as "ACME", "1234", etc.
 2. Verify the accessory does not claim to use any messages it does not use.
 - a. Consult accessory Product Plan for claimed messages.
 - b. Compare to Summary Page in ATS.